



AI-Powered Intelligent Job Search and Career System

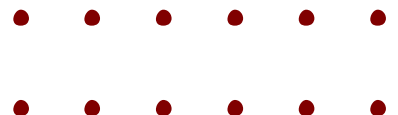
Data Science and AI Project

Milestone 4 Report

Group 4

Prepared By:

Gaurav Kumar (22f1001105)
Dev Gupta (22f2000888)
Abhinav Ohri (24f1002064)
Pranav N (22f2000117)
Praveena N (22f3001454)





Index

SN	Particulars	Page No
1	Introduction	2
2	Datasets Used in This Milestone	3
3	Model Configuration	4
4	Execution Environment	5
5	Implementation Methodology	6
6	Configuration Parameters and Their Selection	7
7	Optimization Methods	8
8	Regularization and Reliability Controls	9
9	Development Progress	10
10	Final Configuration Selection	10
11	Challenges Encountered	11
12	Summary and Next Steps	12
13	Team Consent	12

1. Introduction

Milestone 3 finalised the architecture of the three-module pipeline and the model used at every inference stage. No selection was reversed during Milestone 4 except the Job Discovery judge call has moved from Gemini 2.5 Flash to Gemma 4 31B. The models carried forward are:

Module	Component	Model
Resume Parsing	Primary parser	Gemma 4 31B (multimodal, open-weight)
Resume Parsing	Backup and repair	Gemini 2.5 Flash-Lite
Career Recommendation	Embedding model	gemini-embedding-2, 768 dimensions
Career Recommendation	Vector store	ChromaDB, cosine similarity
Career Recommendation	Explanation model	Gemini 2.5 Flash-Lite
Job Discovery and Matching	Query generation	Gemini 2.5 Flash-Lite
Job Discovery and Matching	Judge	Gemma 4 31B
Job Discovery and Matching	Embedding model	BAAI/bge-base-en-v1.5, 768 dimensions, local CPU
Job Discovery and Matching	Lexical ranking	BM25 (rank_bm25)

1.2 Objectives of this milestone

- Build a working prototype of each of the three modules and confirm that the end-to-end pipeline runs.
- Fix the configuration surface of each module: prompts, output schemas, retrieval parameters, and scoring weights.
- Record the datasets, splits, environment, and constraints under which the prototype operates.
- Identify what remains to be measured, so that Milestone 5 can carry out evaluation against the held-out splits.

1.3 Nature of the work in this milestone

This system performs no gradient-based training. No model weights are initialised, updated, or saved at any point in the pipeline. Every model is pretrained and is used either zero-shot or few-shot through a prompt and an output schema, or as a frozen encoder that produces embeddings. This was a design decision recorded in Milestone 3, taken because the project has no labelled training data of the kind these tasks would require.

The work of this milestone was therefore implementation rather than training. Each module was built, run on development data, and inspected.

2. Datasets Used in This Milestone

2.1 Datasets

Module	Dataset	Role
Resume Parsing	Resume Data (PDF), Kaggle hadikp/resume-data-pdf: 8,905 single-page image-only PDFs, 86-record gold set	Gold set supports prompt iteration and per-field measurement; the remaining corpus supports robustness sampling
Career Recommendation	ESCO v1.2.1 English classification: 3,039 occupations, 13,939 skills, 126,051 occupation-skill relations	Retrieval knowledge base, indexed in full
Job Discovery and Matching	Candidate Job Role Dataset (Kaggle): 1,000 profiles, 200-profile stratified development sample	Query side of the ranking experiments
Job Discovery and Matching	LinkedIn Job Postings Dataset (Kaggle): 123,849 postings, 24,428 companies, 72,521 titles	Offline document pool for ranking experiments
Job Discovery and Matching	Live discovery store (PostgreSQL with pgvector)	Runtime corpus populated by SearXNG discovery and Crawl4AI extraction; not used for offline experiments

2.2 Splits

Module	Split	Detail
Resume Parsing	Development 34, held-out test 52, training 0	Gold set of 86 records stratified across all 43 canonical categories. Near-duplicate groups are confined to a single split so that visually similar resumes cannot straddle the boundary. Random seed 42. The remaining 8,819 unannotated resumes support robustness sampling.
Career Recommendation	Not split	The ESCO occupation set functions as the complete retrieval knowledge base. Withholding occupations would degrade the system rather than create a meaningful held-out condition, since a candidate cannot be matched to an occupation that is absent from the index.
Job Discovery and Matching	60 / 20 / 20 on the 200-profile development sample: 120 / 40 / 40 candidates	Candidate IDs are divided before candidate-job pair construction, so every pair belonging to one candidate stays in the same split. A 60/20/20 ratio was chosen over 80/10/10 because the latter would leave only 20 candidates in each evaluation split, too few for stable ranking estimates.

2.3 Augmentation

Augmentation is used for robustness testing only, never to expand a training set, since there is none.

- **Resume Parsing:** a separate robustness set was generated using small image rotations, moderate Gaussian blur, JPEG compression, contrast reduction, resolution reduction, and light image noise. Each augmented document keeps the annotation and the split of its source resume, so a transformed copy cannot leak across the development and test boundary. Original development and test resumes are left unaltered.
- **Job Discovery and Matching:** positive candidate-job pairs are augmented through controlled skill-synonym replacement using a manually reviewed alias list, two variants per positive pair. Augmentation is applied after splitting and only within the development split; no new skills, qualifications, or job requirements are created.

All models are pretrained and used as supplied. Nothing is trained from scratch and nothing is fine-tuned. The configurable surface of the system is therefore the prompts, the output schemas, the retrieval parameters, and the scoring weights.

3. Model Configuration

3.1 Settings held constant

Setting	Value and reason
Temperature	0 for every generative call. Extraction, explanation, and judging must be reproducible; sampling variety has no value in any of these tasks.
Output format	Schema-constrained JSON on every generative call, validated before the output is accepted downstream.
Model identifiers	Held in environment configuration rather than in source, so a deprecated model can be swapped without a code change.
Random seed	42, fixed across split assignment and sampling.
Embedding dimensions	768 in both Career Recommendation and Job Discovery and Matching.

3.2 Module configuration

Module	Configuration
Resume Parsing	Primary parser: Gemma 4 31B (multimodal, open-weight). Backup and repair: Gemini 2.5 Flash-Lite, used only on fallback. Vision-first routing, with a 100-character-per-page text-layer probe deciding between the text path and the vision path. One page image is sent per call and the per-page JSON is merged afterwards. A single Gemini 2.5 Flash-Lite fallback retry is permitted on schema-validation failure or low field completeness, after which a partial profile with review flags is emitted rather than nothing.
Career Recommendation	Embedding model: gemini-embedding-2, 768 dimensions. Vector store: ChromaDB, cosine similarity. Explanation model: Gemini 2.5 Flash-Lite.

Module	Configuration
	One chunk per occupation, concatenating preferred label, alternative labels, description, and essential skills. Initial vector retrieval of Top-K 20, followed by deterministic Python re-ranking (hard-requirement exclusion, then exact skill-match count, with cosine similarity as tiebreak), reduced to Top-N 5 passed to the explanation model.
Job Discovery and Matching	Query generation: Gemini 2.5 Flash-Lite. Judge: Gemma 4 31B, Embedding model: BAAI/bge-base-en-v1.5, 768 dimensions, local CPU. Lexical ranking: BM25. Seven-stage LangGraph pipeline. Hybrid score = $(0.4 \times \text{BM25} + 0.3 \times \text{embedding}) / 0.7$, keeping the top 15. Final score = $0.8 \times \text{hybrid score} + 0.2 \times \text{judge probability on the top 5}$. Exactly two LLM calls per run regardless of how many jobs are discovered: one query-generation call to Gemini 2.5 Flash-Lite, and one batched judge call to Gemma 4 31B.

4. Execution Environment

Requirement	Resume Parsing	Career Recommendation	Job Discovery and Matching
CPU / GPU	Hosted API only (Gemma 4 31B, Gemini 2.5 Flash-Lite); no local model, no GPU.	Hosted API only (gemini-embedding-2, Gemini 2.5 Flash-Lite) ChromaDB runs on standard CPU.	Hosted API for Gemini 2.5 Flash-Lite and Gemma 4 31B bge-base-en-v1.5 (110M parameters) runs locally on CPU
External services	None beyond the hosted parser API.	ChromaDB persistent local index.	SearXNG (self-hosted) Crawl4AI (self-hosted) Redis (one-hour cache).
Storage	Structured profiles in PostgreSQL Rendered page images are transient and deleted after parsing.	ChromaDB vector index PostgreSQL for recommendation outputs.	PostgreSQL with pgvector for jobs, chunk embeddings, and run records.

4.1 Shared constraints and risks

These apply across all three modules rather than to one of them.

Constraint or risk	Effect	Handling
Free-tier API rate limits	Hosted free tiers used in development are rate-limited to roughly 15 requests per minute and	The architecture already minimises calls to two per Job Discovery run; caching and the global job store

Constraint or risk	Effect	Handling
	1,500 per day for Gemini; the exact figures for whichever service hosts Gemma should be confirmed separately, since they may differ.	further reduce repeated calls during iteration.
No GPU in the current setup	Fine-tuning and locally hosted large models are outside the scope of the pipeline built in this milestone.	All large-model work is hosted, and the only local model is a small CPU encoder. This was a design input for this milestone rather than an obstacle encountered late. GPU capacity may be used in Milestone 5 to accelerate batch inference over the held-out splits.
Vendor model deprecation	Hosted model providers have retired models with short notice in the past, which would break a hard-coded identifier.	All model identifiers are held in environment configuration and can be changed without touching code, across all three modules.
Free-tier PostgreSQL storage	Available storage is limited to roughly 0.3 to 0.5 GB.	The Job Discovery job store is deduplicated by a hash of the normalised posting URL, so storage grows with distinct postings rather than with searches.

5. Implementation Methodology

No weights are trained in this system. Batch size, epochs, and loss function do not apply to any module for the same reason: there is no weight update step for them to govern. What follows is how each module was built and iterated on.

5.1 Resume Parsing

Workflow: Implement the extraction and merge logic, run it over the 34-resume development split, inspect the returned JSON by hand against the source images, adjust the prompt or schema where fields were being missed or invented, and re-run.

Iteration stopped when further changes no longer produced a visible improvement.

One page image is processed per call, which is a routing choice rather than a batch size. Correctness during development was assessed by direct inspection of the returned JSON against the source images; formal per-field Precision, Recall, and F1 on the held-out 52-resume test set is deferred to Milestone 5.

Learning strategy: Zero-shot or Few-shot. A strong instruction-following multimodal model already generalises to resume extraction without task-specific training, so the system avoids the cost of building, labelling for, and maintaining a custom-trained model.

5.2 Career Recommendation

Workflow: Implement retrieval and re-ranking, run it over development candidate profiles, inspect the retrieved occupations and generated explanations by hand, adjust the retrieval parameters or prompt where the output was clearly wrong, and re-run.

There is no dataset passed repeatedly through an optimiser; each candidate profile is processed once. Correctness during development was assessed by inspecting whether retrieved occupations were sensible and whether explanations stayed grounded in the retrieved evidence. Recall@K and similar retrieval metrics are deferred to Milestone 5.

Learning strategy: Retrieval-augmented generation, not training. Training a 3,039-class occupation classifier would require thousands of labelled candidate profiles distributed across every class, which the project does not have and cannot ethically fabricate. Retrieval over ESCO achieves the same goal with no labelled pairs and keeps every recommendation traceable to a specific ESCO record.

5.3 Job Discovery and Matching

Workflow: implement each of the seven stages, run the pipeline against live search, inspect the returned shortlist and rationale by hand, adjust prompts, filters, or weights where the output was clearly wrong, and re-run.

The one place batching appears in this module is an efficiency measure rather than a training parameter: the judge scores all five shortlisted jobs in a single call rather than five separate calls. Correctness during development was assessed by inspecting the returned shortlist and rationale; formal NDCG@10, Precision@20, and Recall@100 measurement is deferred to Milestone 5.

Learning strategy: none. Ranking is computed from BM25, cosine similarity, and a small number of configurable weights; no ranking model is trained.

6. Configuration Parameters and Their Selection

There are no hyperparameters in the training sense. The parameters below are the configurable values that govern system behaviour for each module.

6.1 Resume Parsing

Parameter	Value and basis
Text-layer probe threshold	100 characters per page. Below this a page is treated as image-only and routed to the vision path. The working dataset contains no page with an extractable text layer, so the threshold only affects documents outside it.
Fallback retries	A single retry with a second, independent model recovers most schema failures. Further retries add cost and latency without a corresponding recovery rate.
Pages per call	One page image per request keeps the prompt small and the per-page JSON easy to validate before merging.

6.2 Career Recommendation

Parameter	Value and basis
Initial retrieval Top-K	20. A wider initial set reduces the risk of discarding an unusually titled or hyper-specialised occupation before the deterministic re-ranking step can consider it.
Final Top-N to the explanation model	5. Five occupations fit comfortably in one grounded explanation prompt and match the number of recommendations shown to the user.
Chunking	One occupation per chunk. ESCO is an atomic classification rather than free-form text, so sliding-window chunking would fragment the skill list for a single occupation across entries.

6.3 Job Discovery and Matching

Parameter	Value and basis
BM25 and embedding weights	0.4 and 0.3, normalised by 0.7. BM25 is weighted slightly higher because exact tool and certification names carry strong signal that embeddings can blur.
Shortlist after hybrid ranking	15. Wide enough to absorb ranking error before the judge stage, narrow enough that only 5 finalists need retrieved chunks.
Hybrid and judge weights in the final score	0.8 and 0.2. The deterministic hybrid score is the primary signal. The judge adjusts the ordering rather than dominating it.
RAG chunks per job at judge time	2. Keeps the batched judge prompt small and focused on the most relevant part of each posting.

All four scoring weights are held in environment configuration, so they can be re-tuned by grid search against the validation split in Milestone 5 without any code change.

7. Optimization Methods

Gradient-based optimisers, learning-rate schedules, and gradient clipping do not apply here, since no gradients are computed. The optimisation that did take place was of cost and latency.

7.1 Resume Parsing

No cost or latency optimisation beyond what is already fixed in Section 6.1: one page per call and a single fallback retry. The binding cost for this module is per-page hosted API latency rather than call volume, since exactly one primary call and at most one fallback call are made per page regardless of resume length.

7.2 Career Recommendation

Persistent ESCO index: occupation embeddings are generated once offline and stored in ChromaDB, so no ESCO embedding call is made at request time. This is the module's only distinct cost optimisation, since a candidate query requires exactly one embedding call and one explanation call regardless of how large the ESCO index is.

7.3 Job Discovery and Matching

Optimisation	Effect
Exactly two LLM calls per run	A naive per-item design would issue one extraction call per discovered job and one judge call per shortlisted job, an estimated 45 or more calls per run. Making extraction deterministic and batching the judge reduces this to two calls regardless of how many jobs are found.
Zero-LLM extraction	Structured fields are read from schema.org JobPosting JSON-LD or page metadata rather than from a model call. This is faster, more reliable, and free per job.
Global deduplicated job store	Every posting URL reduces to a stable key. A posting is crawled, parsed, and embedded once, then reused by every subsequent search from any candidate until it goes stale or expires.
Local embedding model	bge-base-en-v1.5 runs on CPU, so candidate and job embeddings cost nothing per call and are unaffected by API rate limits.
Redis result cache	SearXNG results are cached for one hour, so repeated or similar queries during development do not re-issue searches.

8. Regularization and Reliability Controls

Dropout, weight decay, and label smoothing do not apply, as there are no weights to regularise. The equivalent concern here is that a pretrained model may produce output that is fluent but unsupported by the source document.

8.1 Resume Parsing

- **Schema constraints:** every call is bound to `parsed_resume_schema.json`, and validation is a hard gate before any profile is persisted or passed downstream.
- **Visible-only extraction:** the prompt forbids inference or guessing. Missing scalar fields are stored as null and missing lists as empty, rather than being filled from plausible assumption.
- **Duplicate-safe splits:** perceptual near-duplicate groups in the resume corpus are confined to a single split, so a visually identical resume cannot appear on both sides of an evaluation boundary.
- **Category-balanced sampling:** the gold set takes two resumes from each of the 43 canonical categories, so that the largest category, which holds roughly 12 times as many resumes as the smallest, cannot dominate the evaluation.
- **Prompt-injection handling:** resume text is treated as untrusted data, delimited clearly, and cannot alter the system instruction.

8.2 Career Recommendation

- **Schema constraints:** the explanation model's output is bound to a Pydantic response schema, validated before being returned.
- **Evidence grounding:** the explanation model may only discuss occupations present in the retrieved set, and may only name skill gaps drawn from that occupation's own ESCO skill list. Every output URI is validated against the retrieved set.

8.3 Job Discovery and Matching

- **Schema constraints:** the batched judge call returns structured output, validated before being accepted.
- **Permissive filtering:** the hard filter excludes a posting only when it states a requirement the candidate explicitly fails. Vague postings pass through, because a wrongly excluded job cannot be recovered later in the pipeline.
- **Prompt-injection handling:** job descriptions are treated as untrusted data, delimited clearly, and cannot alter the system instruction.

9. Development Progress

There are no loss curves or accuracy-against-epoch plots to report, since nothing is trained. What follows is the state of each module at the end of this milestone.

9.1 Resume Parsing

Status: working prototype. The extraction path runs end to end. A resume is routed by the text-layer probe, rendered page by page where required, sent to the parser with the schema prompt, and the returned JSON is validated, merged across pages, normalised, and deduplicated. The fallback retries path and the partial-profile-with-review-flags path both function.

9.2 Career Recommendation

Status: working prototype. The offline stage is complete: all 3,039 ESCO occupations have been converted to occupation-profile chunks, embedded, and written to the persistent ChromaDB index. The inference stage runs: a candidate profile is flattened into a semantic query, embedded, matched against the index, re-ranked deterministically, and passed to the explanation model, which returns schema-validated output.

9.3 Job Discovery and Matching

Status: working prototype. All seven pipeline stages are implemented and run in sequence against live search. Query generation, SearXNG search with Redis caching, Crawl4AI extraction on a job-store miss, regex hard filtering, hybrid BM25 and embedding ranking, and the batched judge call all function, and the ranked shortlist of five jobs is returned with scores and rationale. The global job store correctly skips crawling for postings already held.

10. Final Configuration Selection

Because no models are trained, there are no checkpoints to compare and no best epoch to select. What is fixed at the end of this milestone is a configuration bundle per module, which is the artefact that Milestone 5 will evaluate.

10.1 Resume Parsing

Element	Fixed value
Model identifiers	Gemma 4 31B (primary parser); Gemini 2.5 Flash-Lite (fallback and repair)
Prompts and schemas	Resume extraction prompt bound to parsed_resume_schema.json
Thresholds	Text-layer probe 100 characters per page; one fallback retry; temperature 0 (Section 1.4)

10.2 Career Recommendation

Element	Fixed value
Model identifiers	gemini-embedding-2 (768 dimensions); Gemini 2.5 Flash-Lite (explanation)
Prompts and schemas	Career explanation prompt bound to its Pydantic response schema
Retrieval parameters	Top-K 20; Top-N 5
Thresholds	Temperature 0 (Section 1.4)

10.3 Job Discovery and Matching

Element	Fixed value
Model identifiers	Gemini 2.5 Flash-Lite (query generation); Gemma 4 31B (judge); BAAI/bge-base-en-v1.5 (embedding)
Prompts and schemas	Query-generation prompt; batched judge prompt with structured output
Retrieval parameters	Shortlist 15; 2 RAG chunks per job at judge time
Scoring weights	BM25 0.4, embedding 0.3, normalisation 0.7; hybrid 0.8, judge 0.2
Thresholds	Temperature 0 (Section 1.4)

11. Challenges Encountered

11.1 Resume Parsing

Challenge	Effect	How it was handled
The gold set is provisional	Only 15 of the 86 gold records were verified by three team members against the source images; the remaining 71 were AI-drafted and not individually checked. Reported field-level results will carry the accuracy of the annotations themselves.	The limitation is stated rather than hidden. Verification of the remaining records is scheduled before the held-out evaluation.
Sparse fields in the corpus	Projects appear in 5 of 86 gold records and certifications in 27, so per-field results for these two fields will rest on very few examples.	Per-field metrics will be reported with support counts, and no overall score will be quoted without them.

11.2 Career Recommendation

Challenge	Effect	How it was handled
No labelled candidate-to-occupation data exists	Retrieval quality cannot yet be scored with Recall@K, Precision@K, or MRR, and no numeric target has been set for the module.	A labelled profile-to-occupation set is being prepared as a Milestone 5 dependency. Until then the module is assessed by inspection, and no retrieval target is claimed.

11.3 Job Discovery and Matching

Challenge	Effect	How it was handled
Crawl reliability	Some job boards render poorly or block automated access, and postings without JSON-LD yield weaker structured fields.	A failed crawl skips that URL rather than failing the run. Restricted pages are excluded rather than bypassed.

11.4 Cross-cutting constraints

Free-tier API rate limits, the absence of a GPU in the current setup, and vendor model deprecation affect all three modules rather than one; these are recorded once, in Section 4.1, rather than repeated here.

12. Summary and Next Steps

12.1 Summary

Milestone 4 produced a working prototype of all three modules and fixed the configuration under which they run. Resume Parsing extracts a validated candidate profile from an image-only resume; Career Recommendation has the full ESCO taxonomy embedded and indexed and returns grounded occupation recommendations; Job Discovery and Matching runs its seven-stage pipeline against live search and returns a ranked shortlist of five jobs with scores and rationale.

No training took place, and none was planned. All models are pretrained and used zero-shot or as frozen encoders, for the reasons recorded in Milestone 3. The configurable surface of the system, meaning prompts, schemas, retrieval parameters, and scoring weights, was set and is now frozen for evaluation.

13. Team Consent

Name	Roll Number	Initial
Gaurav Kumar	22f1001105	GK
Dev Gupta	22f2000888	DG
Abhinav Ohri	24f1002064	AO
Pranav N	22f2000117	PN
Praveena N	22f3001454	PN